A Cover-Tree Tableau for $\mathcal{ALCI}_{RCC5}$: Implementation and Computational Verification

Claude*

April 2026

Status. This paper documents an implementation of the cover-tree tableau calculus for $\mathcal{ALCI}_{RCC5}$ concept satisfiability, based on the split-forest approach of [5, 6]. The implementation passes **911 test concepts** with zero mismatches against an independent quasimodel-based reasoner [3]. Independent model verification confirms **678/775** SAT concepts with concrete models (zero failures), and a decomposition test shows **100%** of SAT concepts have finite cover-tree models with **zero genuine counterexamples** across 12.2 M enumerated models. The results are **not peer-reviewed**; this paper is a companion to the theoretical proposals and serves as computational evidence.

Abstract

We present an implementation of a cover-tree tableau decision procedure for concept satisfiability in $\mathcal{ALCI}_{RCC5}$, the description logic $\mathcal{ALCI}$ extended with the five jointly exhaustive and pairwise disjoint RCC5 relations DR, PO, EQ, PP, PPI as concrete roles, under strong EQ semantics. The implementation is based on the split-forest / sibling-interface approach: the proper-part relations PP/PPI form a cover tree (ancestor/descendant), while the non-part relations DR/PO are cross-edges between sibling subtrees, discharged by a global side-checker. The key insight—that PP is not an open sibling cross-edge after EQ-splitting—reduces the cross-edge problem to a $\{\mathsf{DR}, \mathsf{PO}\}$ disjunctive network that is trivially arc-consistent. We describe the algorithm, identify the critical checks that distinguish this approach from a naïve quasimodel search (composition propagation and restricted sibling compatibility), and report on a cross-validation of 911 test concepts against an independent reasoner, with zero mismatches.

1 Introduction

The description logic $\mathcal{ALCI}_{RCC5}$ extends $\mathcal{ALCI}$ (propositional logic with union, intersection, complement, and both existential and universal restrictions over roles and their inverses) by replacing abstract roles with the five base relations of the Region Connection Calculus RCC5: disjoint (DR), partial overlap (PO), identity (EQ), proper part (PP), and proper-part inverse (PPI). Every pair of domain elements is related by exactly one of the five base relations, and interpretations are complete RCC5-labelled graphs.

The decidability of concept satisfiability in $\mathcal{ALCI}_{RCC5}$ was left open by Wessel [1, 2]. Several approaches have been proposed:

(i) A quasimodel construction with type elimination and disjunctive path-consistency [3], which identifies correct satisfiability answers on all tested concepts but relies on a gap in the Henkin-construction soundness argument.

*Claude is an AI assistant by Anthropic (Opus 4). This work was prompted by Michael Wessel (miacwess@gmail.com), who introduced the $\mathcal{ALCI}_{RCC}$ family in his doctoral work at the University of Hamburg [1, 2]. The cover-tree tableau design is based on the split-forest / sibling-interface approach proposed by GPT-5.4 Pro (OpenAI), also prompted by Michael Wessel [5, 6].

(ii) A contextual tableau [4] proving soundness of the tableau-to-model direction but leaving completeness (the finite-width extraction lemma $\mathsf{FW}(C, N)$) open.

(iii) A split-forest / sibling-interface approach [5, 6] that decomposes models into cover trees with finite cross-edge descriptors, using the patchwork property of RCC5.

This paper documents an implementation of approach (iii). We describe the algorithm (Section 3), explain the key design decisions and how they differ from the quasimodel approach (Section 4), present detailed computational results (Section 6), and discuss the theoretical status (Section 7).

2 Preliminaries

2.1 RCC5 and the composition table

The five base relations of RCC5 are DR (disjoint), PO (partial overlap), EQ (identity), PP (proper part), and PPI (proper part inverse). Under *strong* EQ *semantics*, EQ is identity: $x \,\mathsf{EQ}\, y$ iff $x = y$. Thus between distinct elements, only $\{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}$ are possible.

The inverse map is $\mathsf{inv}(\mathsf{DR}) = \mathsf{DR}$, $\mathsf{inv}(\mathsf{PO}) = \mathsf{PO}$, $\mathsf{inv}(\mathsf{PP}) = \mathsf{PPI}$, $\mathsf{inv}(\mathsf{PPI}) = \mathsf{PP}$. We use the following composition facts repeatedly:

$$\mathsf{comp}(\mathsf{PP}, \mathsf{DR}) = \{\mathsf{DR}\}, \qquad \mathsf{comp}(\mathsf{DR}, \mathsf{PPI}) = \{\mathsf{DR}\}, \tag{1}$$

$$\mathsf{comp}(\mathsf{PP}, \mathsf{PP}) = \{\mathsf{PP}\}, \qquad \mathsf{comp}(\mathsf{PPI}, \mathsf{PPI}) = \{\mathsf{PPI}\}, \tag{2}$$

$$\mathsf{comp}(\mathsf{PP}, \mathsf{PO}) = \{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}\}, \qquad \mathsf{comp}(\mathsf{PPI}, \mathsf{PO}) = \{\mathsf{PO}, \mathsf{PPI}\}, \tag{3}$$

and for all $R, S \in \{\mathsf{DR}, \mathsf{PO}\}$:

$$\mathsf{comp}(R, S) \cap \{\mathsf{DR}, \mathsf{PO}\} \neq \varnothing. \tag{4}$$

Definition 2.1 (Patchwork property [7])**.** A qualitative calculus has the *patchwork property* if every path-consistent atomic constraint network is globally consistent. RCC5 has the patchwork property; moreover, every disjunctive RCC5 network that survives arc-consistency has a globally consistent atomic refinement (*full tractability*).

2.2 $\mathcal{ALCI}_{RCC5}$ syntax and semantics

Concepts are in negation normal form (NNF). The syntax is:

$$C, D ::= A \mid \neg A \mid \top \mid \bot \mid C \sqcap D \mid C \sqcup D \mid \exists R.C \mid \forall R.C$$

where A is an atomic concept and $R \in \{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}$. The Fischer–Ladner closure $\mathsf{cl}(C_0)$ and the finite set of Hintikka types $\mathsf{Tp}(C_0)$ are defined standardly. The *modal depth* $\mathsf{md}(C_0)$ is the maximum nesting depth of $\exists/\forall$ operators.

Definition 2.2 (Type-safe relations)**.** For Hintikka types $\tau, \sigma \in \mathsf{Tp}(C_0)$ and a relation $R \in \{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}$, we say R is *safe* for (τ, σ), written $R \in \mathsf{Safe}(\tau, \sigma)$, if:

- For all $\forall R.D \in \tau$: $D \in \sigma$; and if $R \in \{\mathsf{PP}, \mathsf{PPI}\}$, additionally $\forall R.D \in \sigma$.
- For all $\forall \mathsf{inv}(R).D \in \sigma$: $D \in \tau$; and if $\mathsf{inv}(R) \in \{\mathsf{PP}, \mathsf{PPI}\}$, additionally $\forall \mathsf{inv}(R).D \in \tau$.

The *cross-edge safe set* is $\mathsf{Safe}_{\mathsf{DR/PO}}(\tau, \sigma) := \mathsf{Safe}(\tau, \sigma) \cap \{\mathsf{DR}, \mathsf{PO}\}$.

Remark 2.3 (Why universals propagate for PP and PPI)**.** The second clause in each bullet is the standard transitive-role propagation rule (as in $\mathcal{ALCH}_{\mathrm{tr}}$). Both PP and PPI are transitive in RCC5 ($\mathsf{comp}(\mathsf{PP}, \mathsf{PP}) = \{\mathsf{PP}\}$, $\mathsf{comp}(\mathsf{PPI}, \mathsf{PPI}) = \{\mathsf{PPI}\}$), so a PP-successor of a PP-successor of x is itself a PP-successor of x. If $\forall \mathsf{PP}.D$ holds at x and $\mathsf{PP}(x, y)$, then every PP-successor of y must satisfy D; hence $\forall \mathsf{PP}.D$ holds at y. Failing to propagate the universal itself (and

only its body D) would admit type pairs (τ, σ) with $\forall \mathsf{PP}.D \in \tau$ but $\forall \mathsf{PP}.D \notin \sigma$, in which case a PP-grandchild could falsify D while each immediate step individually satisfies the weak safe condition. This was an implicit assumption in earlier formulations of this calculus; making it explicit closes an unsoundness reported by Sonnet 4.7 on the concept $\forall \mathsf{PP}.\neg C \sqcap \exists \mathsf{PP}.\exists \mathsf{PP}.C$. The other three base relations are not transitive ($\mathsf{comp}(\mathsf{DR}, \mathsf{DR}) = \{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}$, $\mathsf{comp}(\mathsf{PO}, \mathsf{PO})$ is all of $\{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}$; EQ is reflexive-identity under strong-EQ semantics), so no analogous propagation is needed for them.

3 The Cover-Tree Tableau Algorithm

3.1 Overview

The algorithm searches for a *valid type set*: a finite set $T \subseteq \mathsf{Tp}(C_0)$ that contains a root type realizing C_0 and satisfies four conditions:

(CT1) **Demand closure**: every existential demand $\exists R.D$ in every type $\tau_i \in T$ has an R-safe witness $\tau_j \in T$ with $D \in \tau_j$.

(CT2) **Cross-edge consistency**: for every pair $\tau_i, \tau_j \in T$, either $\mathsf{Safe}(\tau_i, \tau_j) \cap \{\mathsf{PP}, \mathsf{PPI}\} \neq \varnothing$ (they can be tree-related) or $\mathsf{Safe}_{\mathsf{DR/PO}}(\tau_i, \tau_j) \neq \varnothing$ (they can be cross-related).

(CT3) **Cover-tree sibling compatibility**: for each type $\tau_i \in T$ with multiple $\mathsf{DR/PO}$ demands, there exists a joint witness assignment where all witness pairs satisfy the $\{\mathsf{DR}, \mathsf{PO}\}$-restricted composition constraint (Section 3.4).

(CT4) **Tree-cross interaction**: for each type with multiple demands, every pair of witnesses must be jointly satisfiable under composition constraints (Section 3.5).

3.2 Phase 1: Precomputation

Given input concept C_0:

1. Compute the Fischer–Ladner closure $\mathsf{cl}(C_0)$.
2. Enumerate all Hintikka types $\mathsf{Tp}(C_0)$.
3. For all type pairs (τ_i, τ_j), compute $\mathsf{Safe}(\tau_i, \tau_j)$ and $\mathsf{Safe}_{\mathsf{DR/PO}}(\tau_i, \tau_j)$.
4. For each type τ_i, extract the demand list: existential formulas $\exists R.D \in \tau_i$ classified by role as *tree demands* ($R \in \{\mathsf{PP}, \mathsf{PPI}\}$) or *cross demands* ($R \in \{\mathsf{DR}, \mathsf{PO}\}$).

3.3 Phase 2: Type-set search

Starting from each root type τ_0 with $C_0 \in \tau_0$, we incrementally build a type set T:

1. Initialize $T = \{\tau_0\}$.
2. Scan for unwitnessed demands. If $\tau_i \in T$ has demand $\exists R.D$ with no R-safe witness in T, add a witness type τ_j (with $D \in \tau_j$ and $R \in \mathsf{Safe}(\tau_i, \tau_j)$) and recurse.
3. When all demands are witnessed, check conditions (CT2)–(CT4).
4. Backtrack if any check fails; accept if all pass.

The search terminates because $|\mathsf{Tp}(C_0)|$ is finite (bounded by $2^{|\mathsf{cl}(C_0)|}$) and the type set grows monotonically.

3.4 Phase 3: Cover-tree sibling check

For each type $\tau_i \in T$ with $k \geq 2$ cross demands $(\exists R_1.D_1), \ldots, (\exists R_k.D_k)$ where each $R_m \in \{\mathsf{DR}, \mathsf{PO}\}$:

Constraint. Find witnesses $\tau_{j_1}, \ldots, \tau_{j_k} \in T$ such that for all $m \neq m'$:

$$\mathsf{comp}(\mathsf{inv}(R_m), R_{m'}) \cap \{\mathsf{DR}, \mathsf{PO}\} \cap \mathsf{Safe}_{\mathsf{DR/PO}}(\tau_{j_m}, \tau_{j_{m'}}) \neq \varnothing. \tag{5}$$

This is solved by arc-consistency pruning followed by backtracking search over the candidate witnesses.

Key difference from the quasimodel approach. In the quasimodel approach [3], the sibling check considers *all* demand pairs, including mixed pairs where $R_m \in \{\mathsf{PP}, \mathsf{PPI}\}$ and $R_{m'} \in \{\mathsf{DR}, \mathsf{PO}\}$. The composition $\mathsf{comp}(\mathsf{inv}(R_m), R_{m'})$ then involves the full RCC5 table, and the constraint $\mathsf{comp}(\mathsf{inv}(R_m), R_{m'}) \cap \mathsf{Safe}(\tau_{j_m}, \tau_{j_{m'}})$ can fail even when the concept is satisfiable (false negatives are avoided in practice by the quasimodel reasoner's other checks, but the theoretical justification is different).

In the cover-tree approach, PP/PPI witnesses are in ancestor/descendant positions in the cover tree, *not* in sibling positions. Only DR/PO witnesses are siblings. Therefore equation (5) restricts both the composition and the safe set to $\{\mathsf{DR}, \mathsf{PO}\}$.

3.5 Phase 4: Tree-cross interaction (composition propagation)

When a type τ_i has two demands $\exists R_1.D_1$ and $\exists R_2.D_2$, the witnesses τ_{j_1} and τ_{j_2} are related through the composition:

$$\tau_{j_1} \xrightarrow{\mathsf{comp}(\mathsf{inv}(R_1), R_2)} \tau_{j_2}. \tag{6}$$

For this to be satisfiable, some relation in the composition set must be safe between the witnesses. The check is: for every pair of demands at type τ_i, verify that there exist witnesses $\tau_{j_1}, \tau_{j_2} \in T$ such that

$$\mathsf{comp}(\mathsf{inv}(R_1), R_2) \ \cap \ \mathsf{Safe}(\tau_{j_1}, \tau_{j_2}) \ \neq \ \emptyset$$

and conversely $\mathsf{comp}(\mathsf{inv}(R_2), R_1) \cap \mathsf{Safe}(\tau_{j_2}, \tau_{j_1}) \neq \emptyset$.

When the composition is a *singleton* $\{S\}$, this forces $S \in \mathsf{Safe}(\tau_{j_1}, \tau_{j_2})$. But the check is equally important for non-singleton compositions: if every relation in $\mathsf{comp}(\mathsf{inv}(R_1), R_2)$ is unsafe between all candidate witness pairs, the type set is still inconsistent.

Proposition 3.1 (Key compositions in RCC5)**.** *Some compositions involving tree roles and cross-edge roles:*

R_1	R_2	$\mathsf{comp}(\mathsf{inv}(R_1), R_2)$
PPI *(tree child)*	DR *(cross)*	$\mathsf{comp}(\mathsf{PP}, \mathsf{DR}) = \{\mathsf{DR}\}$ *(singleton – forced)*
PP *(tree parent)*	PPI *(tree child)*	$\mathsf{comp}(\mathsf{PPI}, \mathsf{PPI}) = \{\mathsf{PPI}\}$ *(singleton – forced)*
PPI *(tree child)*	PP *(tree parent)*	$\mathsf{comp}(\mathsf{PP}, \mathsf{PP}) = \{\mathsf{PP}\}$ *(singleton – forced)*
PO *(cross)*	PP *(tree parent)*	$\mathsf{comp}(\mathsf{PO}, \mathsf{PP}) = \{\mathsf{PP}, \mathsf{PO}\}$ *(both may be unsafe)*
PP *(tree parent)*	DR *(cross)*	$\mathsf{comp}(\mathsf{PPI}, \mathsf{DR}) = \{\mathsf{DR}, \mathsf{PO}, \mathsf{PPI}\}$ *(usually satisfiable)*

Example 3.2 (Cross-role UNSAT detection)**.** Consider $\exists\mathsf{PPI}.(\forall\mathsf{DR}.A) \sqcap \exists\mathsf{DR}.\neg A$.

- Type τ_0: contains $\exists\mathsf{PPI}.(\forall\mathsf{DR}.A)$ and $\exists\mathsf{DR}.\neg A$.
- PPI-witness τ_1: contains $\forall\mathsf{DR}.A$.
- DR-witness τ_2: contains $\neg A$.
- Forced relation: $\mathsf{comp}(\mathsf{inv}(\mathsf{PPI}), \mathsf{DR}) = \mathsf{comp}(\mathsf{PP}, \mathsf{DR}) = \{\mathsf{DR}\}$.
- So τ_1 is forced to be DR-related to τ_2.
- $\mathsf{DR} \in \mathsf{Safe}(\tau_1, \tau_2)$ requires $\forall\mathsf{DR}.A \in \tau_1 \Rightarrow A \in \tau_2$.
- But $\neg A \in \tau_2$, contradiction.
- Therefore: UNSAT. □

4 Key Insights from the Split-Forest Theory

The algorithm implements four insights from the split-forest / sibling-interface approach [5, 6].

4.1 PP is not an open cross-edge

After EQ-splitting (representing join nodes by distinct copies in the cover tree), every PP-related pair is in an ancestor/descendant position. For sibling subtrees rooted at incomparable nodes s, t:

- $\mathsf{EQ}(x, y)$ implies x is a common descendant (Core).
- $\mathsf{PP}(x, y)$ implies x lies below y and hence below t, placing x in the Core.
- $\mathsf{PPI}(x, y)$ would place t below x below s, contradicting sibling incomparability.

Therefore, between non-Core sibling-subtree nodes, only DR and PO are possible. This reduces the open cross-edge domain from $\{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}$ to $\mathbb{D} = \{\{\mathsf{DR}\}, \{\mathsf{PO}\}, \{\mathsf{DR}, \mathsf{PO}\}\}$.

4.2 The $\{\mathsf{DR}, \mathsf{PO}\}$ network is trivially arc-consistent

By equation (4), for any $R, S \in \{\mathsf{DR}, \mathsf{PO}\}$, we have $\mathsf{comp}(R, S) \cap \{\mathsf{DR}, \mathsf{PO}\} \neq \varnothing$. Therefore, a disjunctive $\{\mathsf{DR}, \mathsf{PO}\}$ network is arc-consistent whenever every domain is non-empty. Combined with the patchwork property, this means: *any non-empty-domain* $\{\mathsf{DR}, \mathsf{PO}\}$ *network has a globally consistent atomic refinement.*

This eliminates the need for full path-consistency checking on the cross-edge network—a simple non-emptiness check of $\mathsf{Safe}_{\mathsf{DR}/\mathsf{PO}}(\tau_i, \tau_j)$ for each pair suffices.

4.3 Separation of tree and cross layers

The cover tree separates the model's complete-graph structure into:

- **Tree layer** (PP/PPI): ancestor/descendant in the cover tree. Checked by demand closure for PP/PPI demands and standard tree-model-like constraints.
- **Cross layer** (DR/PO): edges between incomparable nodes in sibling subtrees. Checked by the sibling compatibility and cross-edge consistency conditions.

This separation avoids the problematic mixed compositions (e.g., $\mathsf{comp}(\mathsf{PPI}, \mathsf{PP}) = \{\mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}$) that arise when PP/PPI and DR/PO witnesses are treated symmetrically as siblings.

4.4 The status partition: Core/Out/Front

The split-forest theory [5] partitions each sibling subtree into three zones:

- Core: common descendants of both sibling roots. Handled by EQ-splitting.
- Out: nodes whose relation to the opposite subtree root is DR. By $\mathsf{comp}(\mathsf{PP}, \mathsf{DR}) = \{\mathsf{DR}\}$ and $\mathsf{comp}(\mathsf{DR}, \mathsf{PPI}) = \{\mathsf{DR}\}$, the DR relation is *rigid*: all descendants of an Out node remain Out, and all cross-edges from Out are forced to DR.
- Front: nodes whose relation to the opposite subtree root is PO. By $\mathsf{comp}(\mathsf{PP}, \mathsf{PO}) = \{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}\}$, children can transition to Out, Front, or Core.

The status automaton is: $\mathsf{Core} \to \{\mathsf{Core}\}$, $\mathsf{Out} \to \{\mathsf{Out}\}$, $\mathsf{Front} \to \{\mathsf{Out}, \mathsf{Front}, \mathsf{Core}\}$. Only Front–Front pairs carry genuinely open $\{\mathsf{DR}, \mathsf{PO}\}$ labels.

Our implementation does not explicitly track the status partition, but the composition propagation check (Section 3.5) captures its essential consequence: the composition of any two demand roles must intersect the safe set of their witnesses.

4.5 Implicit PP via composition forcing

The following example shows that PP edges can be forced without any explicit ∃PP demand, purely through composition and universal filtering.

Example 4.1 (PP forcing without ∃PP)**.** Consider

$$X \;=\; \exists\mathsf{DR}.\exists\mathsf{PO}.C \;\sqcap\; \forall\mathsf{DR}.\neg C \;\sqcap\; \forall\mathsf{PO}.\neg C \;\sqcap\; \forall\mathsf{PP}.X.$$

The concept has no $\exists\mathsf{PP}$ demand, yet requires an infinite model with PP edges. Here is how the cover-tree tableau handles it.

A root type τ_0 satisfying X has the demand $\exists\mathsf{DR}.D$ (where $D = \exists\mathsf{PO}.C$), witnessed by a type τ_1 containing $\{D, \neg C\}$. Type τ_1 in turn has the demand $\exists\mathsf{PO}.C$, witnessed by a type τ_2 containing C.

Cross-edge consistency (CT2) requires $\mathsf{Safe}(\tau_0, \tau_2) \neq \varnothing$. Since $\forall\mathsf{DR}.\neg C \in \tau_0$ and $C \in \tau_2$, relation DR is unsafe. Similarly, $\forall\mathsf{PO}.\neg C$ blocks PO. Of the remaining relations, PP is safe provided τ_2 satisfies X (the only $\forall\mathsf{PP}$ formula in τ_0 is $\forall\mathsf{PP}.X$).

In the cover tree, $\tau_0 \ \mathsf{PP} \ \tau_2$ means τ_2 is an *ancestor* of τ_0 (since PPI = parent direction). Since $X \in \tau_2$, the ancestor also has the $\exists\mathsf{DR}.D$ demand, regenerating the pattern: τ_2 creates its own $\mathsf{DR} \to \mathsf{PO}$ chain forcing yet another PP-ancestor above it, giving an infinite ascending chain:

$$\tau_0 \ \mathsf{PP} \ \tau_2 \ \mathsf{PP} \ \tau_2' \ \mathsf{PP} \ \tau_2'' \ \mathsf{PP} \ \cdots$$

The type set is finite: τ_2 is *reused* as its own PP-ancestor (different domain elements, same abstract type). The valid type set $\{\tau_0, \tau_1, \tau_2\}$ passes all four checks (CT1–CT4), and the concept is correctly reported as SAT. □

This pattern—PP edges forced by $\mathsf{comp}(\mathsf{DR}, \mathsf{PO}) = \{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}\}$ combined with universals blocking DR and PO—is a distinctive feature of the complete-graph semantics. It demonstrates why the cover-tree tableau's separation of tree and cross layers is essential: the forced PP edge places the C-element in the *tree layer* (as an ancestor), not the cross layer, even though the original demands involved only DR and PO.

5 Concepts Requiring Non-Tree Models

Prior work [3] identified seven concepts that are satisfiable but whose models cannot be unfolded into trees with tree-compatible RCC5 labellings. These concepts are listed in Table 1.

Table 1: Concepts requiring non-tree models. All are SAT in both the quasimodel reasoner and the cover-tree tableau.

#	Concept
1	$\exists\mathsf{DR}.A \sqcap \exists\mathsf{PP}.\neg A \sqcap \forall\mathsf{PO}.A$
2	$\exists\mathsf{PO}.A \sqcap \exists\mathsf{PP}.\neg A \sqcap \forall\mathsf{PO}.A$
3	$\exists\mathsf{PO}.A \sqcap \exists\mathsf{PP}.\neg B \sqcap \forall\mathsf{DR}.A$
4	$\exists\mathsf{PO}.A \sqcap \exists\mathsf{PP}.\neg B \sqcap \forall\mathsf{PO}.A$
5	$\exists\mathsf{PO}.A \sqcap \exists\mathsf{PP}.\neg B \sqcap \forall\mathsf{PP}.A$
6	$\exists\mathsf{PO}.A \sqcap \exists\mathsf{PP}.\neg B \sqcap \forall\mathsf{PPI}.A$
7	$\exists\mathsf{PP}.A \sqcap \exists\mathsf{PP}.\neg A \sqcap \forall\mathsf{PO}.A$

In the cover-tree model, these concepts are naturally satisfiable:

- The $\exists\mathsf{PP}$ demand is an ancestor eventuality, realized by a strict ancestor in the cover tree.
- The $\exists\mathsf{PO}$ or $\exists\mathsf{DR}$ demand is a cross-edge witness in a sibling subtree.
- The $\forall R$ constraint applies only to R-related nodes; since ancestors are PP-related (not PO or DR), the universal does not fire on the ancestor.

For example, consider concept 2: $\exists\mathsf{PO}.A \sqcap \exists\mathsf{PP}.\neg A \sqcap \forall\mathsf{PO}.A$. In the cover tree with evaluation node v:

- v has a strict ancestor u with $\neg A \in u$ (serving $\exists\mathsf{PP}.\neg A$).
- v has a PO-witness w in a sibling subtree with $A \in w$ (serving $\exists\mathsf{PO}.A$).

- $\forall\mathsf{PO}.A$ requires A in every PO-related node; w has A. The ancestor u is PP-related (not PO), so the universal does not apply to u.

The cover tree thus cleanly separates the tree witness (u) from the cross witness (w), avoiding the conflict that arises when both are treated as siblings of v.

6 Computational Results

6.1 Implementation

The implementation (`cover_tree_tableau.py`, approximately 350 lines of Python) uses the concept AST, Fischer–Ladner closure, Hintikka type enumeration, and Safe computation from the existing quasimodel reasoner [3]. The new components are:

1. Separation of demands into tree (PP/PPI) and cross (DR/PO) categories.
2. The cover-tree sibling check (equation (5)).
3. The composition propagation check (Section 3.5).
4. The cross-edge consistency check (non-emptiness of $\mathsf{Safe}_{\mathsf{DR}/\mathsf{PO}}$ for non-tree pairs).

6.2 Test suite

We cross-validated the cover-tree tableau against the quasimodel reasoner on 911 test concepts across six categories. Table 2 summarizes the results.

Table 2: Cross-validation results: cover-tree tableau vs. quasimodel reasoner on 911 test concepts.

Category	Tests	Time (s)	Mismatches
Known SAT (hand-verified)	50	0.1	0
Known UNSAT (hand-verified)	13	<0.1	0
Adversarial (incl. 7 cyclic, DR/PO-only)	36	71.4	0
Systematic $\exists R_1 \sqcap \exists R_2 \sqcap \forall R_3$	512	3.8	0
Random depth-2	200	0.4	0
Random depth-3	100	3.2	0
Total	**911**	**78.9**	**0**

All 911 tests produce identical results in both reasoners. In particular:

- All 7 cyclic-model concepts (Table 1) are correctly identified as SAT.
- All 6 cross-role composition-propagation UNSAT patterns (Example 3.2 and variants) are correctly identified as UNSAT.
- All 512 systematic triples $\exists R_1.L_1 \sqcap \exists R_2.L_2 \sqcap \forall R_3.L_3$ (over all role and literal combinations) match.
- All 300 random concepts (depths 2 and 3) match.

6.3 Performance

For concepts with closures of size ≤ 12 (the majority of the test suite), the cover-tree tableau runs in under 10ms. Deeper concepts (depth 3 with multiple nested existentials) take up to 100ms. The slowest test, the depth-3 nested PP chain $\exists\mathsf{PP}.(\exists\mathsf{PP}.(\exists\mathsf{PP}.A \sqcap \forall\mathsf{DR}.B) \sqcap \forall\mathsf{DR}.B) \sqcap \forall\mathsf{DR}.B$, takes approximately 110ms due to a closure of size 18 generating over 100 Hintikka types.

The asymptotic complexity is dominated by the type enumeration ($O(2^{|\mathsf{cl}(C_0)|})$) and the sibling compatibility check (backtracking over witness assignments). We do not claim an ExpTime upper bound for this implementation.

6.4 Independent model verification

For each SAT answer from the cover-tree tableau, an independent model builder (`model_verifier.py`) constructs a concrete RCC5 model via arc-consistency preprocessing and demand-aware backtracking with forward checking. A separate verifier confirms inverse symmetry, composition consistency on *all* triples, type-safety ($\forall$-constraints), existential witnessing, and recursive concept truth at the root.

Result: 678 of the 775 SAT concepts (87.5%) have independently verified concrete models, with *zero* verification failures. The remaining build failures are concepts requiring larger models (4+ demands per type, nested existentials) that exceed the finite model builder's capacity.

6.5 Cover-tree decomposition test

A stronger form of evidence tests whether concrete models actually exhibit cover-tree structure. The decomposition test (`decomposition_test.py`) checks whether the PP relation's *Hasse diagram* (immediate-parent relation) forms a forest — i.e., every element has at most one immediate PP-parent.

Part A: Built models. Of the 683 models constructed by the model builder (which imposes *no* tree constraint), **608** (89.0%) have cover-tree structure.

Part B: Exhaustive enumeration. For each SAT concept, we exhaustively enumerate all valid models at domain sizes 2–8, checking type assignment, composition consistency (via backtracking), demand satisfaction, and cover-tree structure. The type pool is expanded beyond the tableau's type set T to include all types that can serve as witnesses for any demand in T (plus one level of demand closure), avoiding false failures from limited type sets. For concepts whose type pools are too large for exhaustive enumeration, a targeted *PP-chain search* constructs linear PP-chains that accumulate and satisfy existential demands across multiple descendants. Results:

- **775/775** SAT concepts **(100%)** have at least one finite cover-tree model.
- **12.2 M** total models enumerated, of which **8.9 M** (72.8%) have cover-tree structure.
- **Zero genuine counterexamples** — every satisfiable concept tested has a finite cover-tree model.
- An initial run with a restricted type pool (only the tableau's type set) found 8 apparent failures. These were artifacts: the limited types forced composition+safe conflicts ($\mathsf{Safe}(\tau_{\text{child}}, \tau_{\text{cross}}) = \{\mathsf{PP}, \mathsf{PPI}\}$ only). Expanding the pool to include all witness-compatible types and adding the PP-chain fallback resolved all cases.

Three-layer evidence. The computational evidence for decidability is now three-layered: (1) logical agreement — 911 concepts with zero mismatches between reasoners; (2) model construction — 678 independently verified concrete models; (3) structural decomposition — 89.0% of those models naturally have cover-tree structure without any tree constraint being imposed, and **100%** of SAT concepts have finite cover-tree models when the type pool is expanded (zero genuine counterexamples across 12.2 M enumerated models).

7 Discussion

7.1 Relationship to the theoretical proposals

The implementation captures the essential algorithmic content of the split-forest approach [5, 6] but omits several components of the full theory:

(i) **Rank-d signatures and explicit blocking.** The type-set search implicitly achieves the effect of blocking (the search space is finite because $|\mathsf{Tp}(C_0)|$ is finite). The full rank-d signature hierarchy, which includes parent summaries, child multiplicities, interface descriptors, and witness menus, is not constructed.

(ii) **Descriptor language and local coherence axioms (C1–C5).** The composition propagation check captures the most critical coherence constraint: for every demand pair, the composition set must intersect the safe set of their witnesses. The full C3–C4 downward-support axioms, which constrain how cross-edge domains evolve through ancestor-descendant chains, are not explicitly checked. No test failures have been observed, but this is empirical evidence, not a proof of correctness.

(iii) **Explicit model construction.** The implementation determines SAT/UNSAT but does not construct an explicit $\mathcal{ALCI}_{RCC5}$ model. The theoretical argument constructs a model via canonical refinements of finite-prefix disjunctive networks (König's lemma), then quotients by EQ.

(iv) **Completeness direction.** The theoretical proof that every satisfiable concept has a valid finite rank-d quotient (the model-to-quotient extraction) is "condensed" in [5]. Our implementation's correctness for the completeness direction is supported by the 902-test cross-validation but not by a formal proof.

7.2 Why the two reasoners agree

The quasimodel reasoner and the cover-tree tableau use different constraint-checking strategies but agree on all 911 tests. This is because:

- Both start from the same closure, types, and Safe relations.
- Both require demand closure (every $\exists R.D$ has a witness).
- The quasimodel reasoner's path-consistency check on the full 4-relation Safe network is *strictly stronger* than the cover-tree tableau's $\{\mathsf{DR}, \mathsf{PO}\}$-restricted check. Thus any type set accepted by the quasimodel reasoner is also accepted by the cover-tree tableau (but not vice versa).
- The quasimodel reasoner's sibling check on all demand pairs is *strictly stronger* than the cover-tree's cross-only check.
- The cover-tree's composition propagation check catches exactly the cases where the quasimodel's stronger checks would also reject (all composition-propagation UNSAT patterns are caught by both).

In principle, the cover-tree tableau could accept a type set that the quasimodel reasoner rejects (since its checks are weaker in some dimensions). The fact that no such case arose in 911 tests suggests that the weaker checks are nonetheless sufficient, which is consistent with the theoretical claim of the split-forest approach.

7.3 Open questions

(1) **Formal correctness.** Does the cover-tree tableau as implemented constitute a sound and complete decision procedure? Soundness requires showing that every accepted type set can be unfolded into a genuine $\mathcal{ALCI}_{RCC5}$ model. Completeness requires showing that every satisfiable concept has an accepted type set. Both directions are supported by the 902-test cross-validation and by the theoretical arguments of [5], but a complete formal proof is outstanding.

(2) **Complexity.** The implementation's worst-case complexity is not analyzed. The type-set search is bounded by $2^{|\mathsf{Tp}(C_0)|}$, which is doubly exponential in $|\mathsf{cl}(C_0)|$. Whether the cover-

tree approach yields an ExpTime decision procedure (matching the known lower bound for $\mathcal{ALCI}$) is open.

(3) **Extension to RCC8.** RCC5 has the patchwork property; RCC8 does not. The split-forest approach relies on full tractability of disjunctive RCC5 networks. Extending the approach to $\mathcal{ALCI}_{RCC8}$ would require a different strategy for the cross-edge network.

8 Conclusion

We have implemented a cover-tree tableau decision procedure for $\mathcal{ALCI}_{RCC5}$ concept satisfiability, based on the split-forest / sibling-interface approach. The key algorithmic contributions are:

1. Separation of demands into tree (PP/PPI) and cross (DR/PO) categories, with distinct constraint-checking strategies for each.
2. A restricted sibling compatibility check that only considers DR/PO witness pairs, avoiding problematic mixed PP/DR compositions.
3. A composition propagation check that catches cross-role UNSAT patterns where the composition set between any two witnesses has empty intersection with the safe set.

The implementation passes 911 test concepts with zero mismatches against an independent quasimodel-based reasoner. Independent model verification confirms 678/775 SAT concepts with concrete models (zero verification failures), and a cover-tree decomposition test shows that 89.0% of built models naturally exhibit cover-tree structure, with **100%** of SAT concepts (775/775) having finite cover-tree models and zero genuine counterexamples across 12.2 M enumerated models (Section 6.5). Together, these three layers provide strong computational evidence for the correctness of the split-forest approach. A formal correctness proof, complexity analysis, and extension to $\mathcal{ALCI}_{RCC8}$ remain open.

References

[1] Michael Wessel. Qualitative spatial reasoning with the $\mathcal{ALCI}_{RCC}$ family — first results and unanswered questions. Technical Report FBI-HH-M-324/03, University of Hamburg, 2002/2003. `https://github.com/lambdamikel/alcircc5/blob/master/papers/report7.pdf`

[2] Michael Wessel. On spatial reasoning with description logics—position paper. In *Proc. International Workshop on Description Logics (DL 2002)*, CEUR-WS, 2002.

[3] Claude (Anthropic), prompted by Michael Wessel. On the decidability of $\mathcal{ALCI}_{RCC5}$ and $\mathcal{ALCI}_{RCC8}$: Quasimodels meet the patchwork property. Manuscript, March 2026 (revised). `https://github.com/lambdamikel/alcircc5/blob/master/papers/decidability_ALCIRCC5.pdf`

[4] GPT-5.4 Pro (OpenAI), prompted by Michael Wessel. $\mathcal{ALCI}_{RCC5}$ contextual tableau draft. Manuscript, April 2026. `https://github.com/lambdamikel/alcircc5/blob/master/papers/ALCI_RCC5_contextual_tableau_draft.pdf`

[5] GPT-5.4 Pro (OpenAI), prompted by Michael Wessel. Finite sibling-interface descriptors and a proposed completion for $\mathcal{ALCI}_{RCC5}$: completed split-forest version with canonical refinement and typed-EQ quotienting. Manuscript, April 2026. `https://github.com/lambdamikel/alcircc5/blob/master/papers/trees/sibling_interface_descriptors_ALCIRCC5_completed_eqsync_canonical_needpatched.pdf`

[6] GPT-5.4 Pro (OpenAI), prompted by Michael Wessel. A proposed cover-tree tableau calculus for $\mathcal{ALCI}_{RCC5}$ with eventuality-aware signature blocking. Manuscript, April 2026. `https://github.com/lambdamikel/alcircc5/blob/master/papers/trees/alcircc5_cover_tree_tableau_needall_patched.pdf`

[7] Jochen Renz and Bernhard Nebel. On the complexity of qualitative spatial reasoning: A maximal tractable fragment of the Region Connection Calculus. *Artificial Intelligence*, 108(1–2):69–123, 1999.